

D_∞ in the Calculus of Inductive Constructions

Lars Warren Ericson¹

¹Catskills Research Company

¹https://github.com/catskillsresearch/scott_models

¹lars.ericson@catskillsresearch.com

August 13, 2026

ORCID: 0000-0001-8299-9361

Abstract

Scott’s reflexive domain D_∞ (the domain-theoretic λ -model with $D_\infty \cong [D_\infty \rightarrow D_\infty]$) can be axiomatized in five layers: **L** (logic), **R** (inference), **S** (capped set theory), **O** (general point-set topology), **D** (domain-theoretic residue). This note states those 20 axioms and 2 rules; then states Lean 4’s kernel calculus (CIC); then gives a faithful deep embedding of L/R/S/O/D in CIC without Mathlib, without `Classical.choice`, and with the specialization order $\sqsubseteq$ recovered as a definition from open sets. Companion artifact: `LRSODInCIC.lean` (standalone Lake package in this directory).

Scope. This is a tutorial exercise in foundational bookkeeping — how few principles suffice, and what each one costs when transcribed into a proof assistant. It is deliberately self-contained: no Mathlib dependency, no library to integrate with, and nothing here is intended as production formalization infrastructure.

Contents

1	Part I — L, R, S, O, D	3
1.1	L — Logical axioms	3
1.2	R — Inference rules	3
1.3	S — Set theory (Z minus Choice minus Replacement, capped at $V_{\omega+2}$)	3
1.4	O — Point-set topology (general)	4
1.5	D — D_∞ -specific axioms	4
1.6	Count	4
2	Part II — CIC (Lean 4’s kernel)	5
2.1	0. Universes	5
2.2	1. Terms	5
2.3	2. Judgments	5
2.4	3. Typing rules	5
2.5	4. Definitional equality	6
2.6	5. Inductive types (generic schema)	6
2.7	6. Quotient types	6
2.8	7. Kernel postulates	7
2.9	Count	7
2.10	Relation to LRSOD (interpretability, not containment)	7

3	Part III — LRSOD in CIC	7
3.1	Vocabulary: <code>structure</code> , <code>class</code> , field, parameter	7
3.2	Layer L, R — logic and rules	9
3.3	Layer S — <code>SetTheory</code>	9
3.4	Layer O — <code>TopologicalSpace D</code>	9
3.5	Layer D — <code>DInfinityFoundations D</code>	10
3.6	A worked witness: the Sierpiński domain	10
3.7	Master correspondence	11
3.8	Mathlib idiom vs deep embedding: <code>structure</code> or <code>class</code> ?	11
3.8.1	O is a class in Mathlib	12
3.8.2	S stays a structure	12
3.8.3	D would be split into Prop mixins, not bundled	12
3.8.4	Why this document keeps <code>structure</code> anyway	13
3.9	Constructivity note (Lean practice vs LRSOD)	14
4	References	14
A	Complete Lean source	14

1 Part I — L, R, S, O, D

Five layers, each built on the previous. **L** and **R** together are logic. **S** is set theory. **O** is *all* of point-set topology — three axioms, no more. **D** is what $D\infty$ needs beyond O, stated entirely in O's vocabulary (τ); the order $\sqsubseteq$ is a *definition*, not a primitive.

1.1 L — Logical axioms

Primitive symbols: variables, $\neg$, $\rightarrow$, $\forall$, $=$, parentheses. Defined: $\wedge, \vee, \leftrightarrow, \exists$ as usual abbreviations.

#	Name	Schema
L1	Propositional 1	$\varphi \rightarrow (\psi \rightarrow \varphi)$
L2	Propositional 2	$(\varphi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \chi))$
L3	Propositional 3	$(\neg\psi \rightarrow \neg\varphi) \rightarrow (\varphi \rightarrow \psi)$
L4	Instantiation	$\forall x \varphi(x) \rightarrow \varphi(t), t \text{ free for } x$
L5	Quantifier distribution	$\forall x(\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \forall x\psi), x \text{ not free in } \varphi$
L6	Equality reflexivity	$x = x$
L7	Equality substitution	$x=y \rightarrow (\varphi(x) \rightarrow \varphi(y)), \varphi \text{ atomic}$

L = 7 axiom schemas.

1.2 R — Inference rules

#	Name	Rule
R1	Modus Ponens	from φ and $\varphi \rightarrow \psi$, infer ψ
R2	Generalization	from φ , infer $\forall x\varphi$

R = 2 rules.

1.3 S — Set theory (Z minus Choice minus Replacement, capped at $V_{\omega+2}$)

Added vocabulary: primitive $\in$. Defined: $\subseteq, \{x, y\}, \bigcup x$, successor, $\emptyset, \omega$ — in the usual way, licensed by S2–S5.

#	Name	Statement
S1	Extensionality	$\forall x \forall y (\forall z (z \in x \leftrightarrow z \in y) \rightarrow x = y)$
S2	Pairing	$\forall x \forall y \exists p \forall z (z \in p \leftrightarrow (z = x \vee z = y))$
S3	Union	$\forall x \exists u \forall z (z \in u \leftrightarrow \exists y (y \in x \wedge z \in y))$
S4	Infinity	$\exists w (\emptyset \in w \wedge \forall y (y \in w \rightarrow S y \in w))$
S5	Separation (Δ_0 schema)	$\forall x \exists s \forall z (z \in s \leftrightarrow (z \in x \wedge \varphi(z))), \varphi \text{ bounded}$
S6	Power Set — single instance, of ω only	$\exists P \forall y (y \in P \leftrightarrow y \subseteq \omega)$

No Choice. No Replacement beyond S5. No Foundation. No further Power Set.

S = 6 axioms/schemas.

1.4 O — Point-set topology (general)

Added vocabulary: unary predicate τ on subsets of a set D (“is open”). D is any set given by S .

#	Name	Statement
O1	Trivial opens	$\tau(\emptyset) \wedge \tau(D)$
O2	Arbitrary unions	$\forall \mathcal{U} \subseteq \mathcal{P}(D) \ [(\forall U \in \mathcal{U}, \tau(U)) \rightarrow \tau(\bigcup \mathcal{U})]$
O3	Finite intersections	$\forall U \forall V [\tau(U) \wedge \tau(V) \rightarrow \tau(U \cap V)]$

Every standard topological notion is a *definition* on top of O1–O3:

$\text{closed}(C) := \tau(D \setminus C)$
 $\text{cl}(A) := \bigcap \{C : \text{closed}(C) \wedge A \subseteq C\}$
 $\text{compact}(K) := \text{every open cover of } K \text{ has a finite subcover}$
 $\text{continuous}(f: D \rightarrow D') := \forall U (\tau'(U) \rightarrow \tau(f^{-1}(U)))$

O = 3 axioms.

1.5 D — D_∞ -specific axioms

O alone cannot supply bottom elements, directed suprema, or a compact-open basis (discrete and indiscrete topologies satisfy O1–O3 while violating D1 and D3). The following are *asserted* in O’s vocabulary only. The specialization order is a definition:

$x \sqsubseteq y := \forall U (\tau(U) \wedge x \in U \rightarrow y \in U)$

#	Name	Statement	Role
D1	T0 (Kolmogorov)	$\forall x \forall y [(\forall U (\tau(U) \rightarrow (x \in U \leftrightarrow y \in U))) \rightarrow x = y]$	$\sqsubseteq$ antisymmetric
D2	Sobriety	every irreducible closed C satisfies $\exists ! x (C = \text{cl}(\{x\}))$	$\sqsubseteq$ directed-complete
D3	Countable compact-open basis	$\exists$ countable basis $B \subseteq \tau$, every $K \in B$ compact, B closed under finite intersection	$\sqsubseteq$ ω -algebraic
D4	Generic bottom point	$\exists \perp \in D \ \forall U (\tau(U) \wedge \perp \in U \rightarrow U = D)$	$\sqsubseteq$ pointed

D1–D4 are independent of O. In L+R+S alone one proves the equivalence with the order-first presentation:

$$(O \wedge D1 \wedge D2 \wedge D3 \wedge D4) \iff \exists \sqsubseteq \ [T1 \wedge \dots \wedge T6 \wedge \tau = \text{Scott}(\sqsubseteq)]$$

Given D1–D4, the tower $D_0, D_1 = [D_0 \rightarrow D_0], \dots$, the inverse limit D_∞ , and $D_\infty \cong [D_\infty \rightarrow _c D_\infty]$ are derived — definitions and theorems, no further axioms.

D = 4 axioms.

1.6 Count

Layer	Axioms/schemas	Rules
L — Logic	7	—
R — Inference	—	2
S — Set theory	6	—
O — Point-set topology	3	—

Layer	Axioms/schemas	Rules
D — D ∞ -specific	4	—
Total	20	2

2 Part II — CIC (Lean 4’s kernel)

Lean 4 implements a dependent type theory — a variant of the **Calculus of Inductive Constructions** with universes, inductive types, and quotient types. There is no separate stratification “logic, then set theory”; typing, term formation, and proof are one mechanism (Curry–Howard). A proposition $P : \mathbf{Prop}$ is proved exactly when some term e is exhibited with $\Gamma \vdash e : P$.

2.1 0. Universes

$u, v, w ::= 0 \mid u+1 \mid \max u \ v \mid \text{imax } u \ v \mid \text{uparam}$

$\text{Sort } 0 = \mathbf{Prop}$ (impredicative). $\text{Sort } (u+1) = \mathbf{Type } u$. $\text{imax } u \ v$ is 0 if $v=0$, else $\max u \ v$ — this makes $\mathbf{Prop}$ impredicative while other universes are predicative.

2.2 1. Terms

$e, A, B ::= \text{Sort } u \mid x \mid c.\{u_1, \dots, u_n\} \mid e_1 \ e_2$
 $\mid \text{fun } x : A \Rightarrow e \mid (x : A) \rightarrow B \mid \text{let } x : A := v; e$
 $\mid C.\text{rec} \mid \text{lit}$

Non-dependent $A \rightarrow B$ is $(x:A) \rightarrow B$ with x not free in B . Connectives $\forall, \exists, \wedge, \vee, \neg, \leftrightarrow$ are library inductives or Pi types, not L-primitives.

2.3 2. Judgments

Judgment	Meaning
$\Gamma \text{ ctx}$	well-formed context
$\Gamma \vdash e : T$	e has type T
$\Gamma \vdash A \equiv B$	definitional equality

2.4 3. Typing rules

#	Name	Rule
K1	Sort	$\Gamma \vdash \text{Sort } u : \text{Sort } (u+1)$
K2	Variable	$x:A \in \Gamma \implies \Gamma \vdash x : A$
K3	Pi-formation	$\Gamma \vdash A : \text{Sort } u, \Gamma, x:A \vdash B : \text{Sort } v \implies \Gamma \vdash (x:A) \rightarrow B : \text{Sort } (\text{imax } u \ v)$
K4	Lambda	$\Gamma, x:A \vdash b : B \implies \Gamma \vdash (\text{fun } x:A \Rightarrow b) : (x:A) \rightarrow B$
K5	Application	$\Gamma \vdash f : (x:A) \rightarrow B, \Gamma \vdash a : A \implies \Gamma \vdash f \ a : B[a/x]$

#	Name	Rule
K6	Let	$\Gamma \vdash v:A, \Gamma, x:A:=v \vdash b:B \implies \Gamma \vdash (\text{let } x:A:=v; b) : B[v/x]$
K7	Conversion	$\Gamma \vdash e:A, \Gamma \vdash A \equiv B, \Gamma \vdash B:\text{Sort } u \implies \Gamma \vdash e:B$

K3's `imax` subsumes L4/L5: a proposition may quantify over an arbitrarily large type and remain in `Prop`.

2.5 4. Definitional equality

#	Name	Rule
E1–E3	Equivalence	reflexivity, symmetry, transitivity
E4	Congruence	for application, lambda, Pi
E5	β	$(\text{fun } x:A \Rightarrow b) a \equiv b[a/x]$
E6	η	$(\text{fun } x:A \Rightarrow f x) \equiv f$
E7	δ	constant unfolds to definition
E8	ζ	let-reduction
E9	ι	recursor-on-constructor
E10	Proof irrelevance	$p, q : P : \text{Prop} \implies p \equiv q$

E10 has no counterpart in L; it is specific to `Prop`.

2.6 5. Inductive types (generic schema)

#	Component	Description
I1	Formation	strict-positive type former <code>C</code>
I2	Introduction	constructors <code>ctor_i</code>
I3	Elimination	recursor <code>C.rec</code>
I4	Computation	ι -rule per constructor

Eq (L6–L7):

```
inductive Eq {α : Sort u} : α → α → Prop where
| refl (a : α) : Eq a a
```

Connectives — ordinary inductives:

```
inductive And (a b : Prop) : Prop where | intro : a → b → And a b
inductive Or (a b : Prop) : Prop where | inl : a → Or a b | inr : b → Or a b
inductive False : Prop where
def Not (a : Prop) : Prop := a → False
```

2.7 6. Quotient types

#	Name	Rule
Q1	Formation	$\alpha:\text{Sort } u, r:\alpha \rightarrow \alpha \rightarrow \text{Prop} \implies \text{Quot } r : \text{Sort } u$
Q2	Constructor	$\text{Quot.mk } r : \alpha \rightarrow \text{Quot } r$

#	Name	Rule
Q3	Lift	<code>Quot.lift f h : Quot r → β</code>
Q4	Computation	<code>Quot.lift f h (Quot.mk r a) ≡ f a</code>
Q5	Soundness	<code>r a b → Quot.mk r a = Quot.mk r b</code>

2.8 7. Kernel postulates

#	Name	Statement
AX1	Propositional extensionality	<code>propext : (a ↔ b) → a = b</code>
AX2	Choice	<code>Classical.choice : Nonempty α → α</code>
AX3	Quotient soundness	<code>Quot.sound</code>

Excluded middle is *not* primitive; `Classical.em` is derived from AX1 + AX2. Function extensionality is a *theorem* from quotients, not a fourth axiom.

2.9 Count

Section	Items
Typing (K1–K7)	7
Definitional equality (E1–E10)	10
Inductive schema (I1–I4)	4
Quotients (Q1–Q5)	5
Postulates (AX1–AX3)	3
Total	29

Only **3** of these are postulates in the LRSOD sense. The other 26 are kernel rules checked mechanically — closer to **R** than to L, S, O, or D.

2.10 Relation to LRSOD (interpretability, not containment)

CIC can *host a model* of L+R+S+O+D by constructing suitable types and structures inside it. That is interpretability, not literal sub-theory inclusion. Lean’s ambient foundation is **stronger** than S: unrestricted universes iterate power types without bound; `Classical.choice` exceeds S’s choice-free regime. A faithful transcription therefore uses a **deep embedding** over an abstract carrier rather than Mathlib’s `Set/TopologicalSpace` used naïvely with `Classical` open. D1–D4 remain explicit hypotheses in any D ∞ formalization — CIC’s expressiveness does not collapse D into definitions.

3 Part III — LRSOD in CIC

3.1 Vocabulary: structure, class, field, parameter

Four words are used throughout this part, and the **structure-versus-class** discussion below turns on the last two. In Lean:

A **structure** is a record type — a labelled bundle of things that must be supplied together.

A **class** is the same kind of record type, declared with the keyword **class** instead, plus one added permission: Lean may supply the bundle **automatically**. A value is registered with **instance**, and requested by writing the type in square brackets, `[TopologicalSpace D]`, rather than by name. Anything a **structure** can hold a **class** can hold too; what differs is *who produces the value* — you, explicitly, or Lean’s instance search, silently.

A **field** is one labelled slot *inside* the bundle, written after **where**.

A **parameter** is an input written in the declaration *header*, before **where**, that must be fixed before the type can even be named.

Side by side, from this document’s own code:

```
-- `D` is a PARAMETER: it sits in the header, outside the bundle.
-- `isOpen`, `openEmpty`, ... are FIELDS: slots inside the bundle.
structure TopologicalSpace (D : Type) where
  isOpen      : (D → Prop) → Prop
  openEmpty   : isOpen (fun _ => False)
  ...

-- No parameters at all. Here the carrier `V` is itself a FIELD.
structure SetTheory where
  V      : Type
  mem    : V → V → Prop
  ...
```

The practical difference is *when* the carrier gets chosen.

`TopologicalSpace` is not a type on its own; you must say `TopologicalSpace Nat` or `TopologicalSpace D`. The carrier is chosen **first, from outside**, and the bundle then describes a topology on that already-fixed type.

`SetTheory` *is* a complete type on its own. The carrier arrives **inside** each value, so two values $T_1\ T_2 : \text{SetTheory}$ may have entirely unrelated carriers $T_1.V$ and $T_2.V$, and one can quantify over all of them with an ordinary $\forall T : \text{SetTheory}, \dots$

A one-line analogy: a parameter is what is printed on the *outside* of the box, so you can ask for that box by name; a field is what is *inside* the box, invisible until you open it.

The **structure/class** difference shows up at every use site — whether the bundle has to be mentioned:

```
-- As declared here (a `structure`): the bundle is a named value, passed by hand.
variable (D : Type) (T : TopologicalSpace D)
example (U : D → Prop) : Prop := T.isOpen U          -- `T` must be written out

-- Sketch, had it been declared with `class`: the bundle is found by search.
variable (D : Type) [TopologicalSpace D]
example (U : D → Prop) : Prop := isOpen U           -- no bundle to write out
```

This matters for **class** because instance search — Lean’s mechanism for silently filling in `[TopologicalSpace D]` — searches by what is on the outside. It can be asked “find me the topology on D ” only because D is a parameter. It cannot be asked to find a bundle whose carrier is hidden inside as a field, since there is nothing to search *by*. Hence the rule that follows: carrier as parameter suggests **class**, carrier as field forces **structure**.

Strategy. Each layer becomes a **structure** whose fields are exactly that layer’s axioms, over abstract carriers. No `import Mathlib`. No `Classical.choice`. Existential set-theoretic axioms (S2, S3, S6) become **operations with characterizing laws**, so downstream proofs never need `Exists.choose`. Separation’s schema becomes `sep : (V → Prop) → V → V` — higher-order over the fixed carrier V , faithful to Δ_0 -in- $\in$ on the capped universe. The order `leq` is defined from `isOpen`, never re-declared as a primitive.

Artifact. `LRSODInCIC.lean` in this directory (standalone Lake package; `cd LRSODInCIC && lake build`).

3.2 Layer L, R — logic and rules

Bare CIC is intuitionistic; L3 (contraposition) is not built in. The embedding adds a local classical axiom weaker than full Choice:

```
axiom lem : ∀ (p : Prop), p ∨ ¬p
```

```
theorem L3 (p q : Prop) : (¬q → ¬p) → (p → q) := by
  intro h hp
  cases lem q with
  | inl hq => exact hq
  | inr hnq => exact absurd hp (h hnq)
```

LRSOD	CIC
L1, L2, L4, L5	K3–K5, I1–I4 (And, Or, Pi = $\forall$)
L3	derived from lem (not from Classical.choice)
L6, L7	Eq, Eq.refl, Eq.subst / Eq.rec
R1	function application (K5)
R2	lambda abstraction (K4)

3.3 Layer S — SetTheory

Subsets of V are predicates $V \rightarrow \text{Prop}$. Membership is $\text{mem} : V \rightarrow V \rightarrow \text{Prop}$.

LRSOD	CIC field
S1 Extensionality	ext
S2 Pairing	pair, pairAx
S3 Union	union, unionAx
S4 Infinity	omega0, infOmega
S5 Separation	sep, sepAx
S6 Power(ω)	powOmega, powOmegaAx

Derived (no new axioms): sub, isInductive, and ω via the Zermelo intersection trick on omega0:

```
def omega : T.V :=
  T.sep (fun z => ∀ I, T.sub I T.omega0 → T.isInductive I → T.mem z I) T.omega0
```

3.4 Layer O — TopologicalSpace D

Open sets are predicates $D \rightarrow \text{Prop}$; isOpen is the sole primitive.

LRSOD	CIC field
O1	openEmpty, openFull
O2	openUnion (indexed by any $\iota : \text{Type}$)
O3	openInter

Derived definitions (no new axioms):

Notion	CIC
$x \sqsubseteq y$	$\text{leq } x \ y := \forall U, \text{isOpen } U \rightarrow U \ x \rightarrow U \ y$

Notion	CIC
closed	<code>isClosed C := isOpen (fun x => ¬ C x)</code>
closure singleton	<code>clSingleton y x := leq x y</code>
compact	finite subcover via <code>Fin n</code> (no Mathlib <code>Finset</code>)
irreducible	<code>irreducible C</code>

3.5 Layer D — DInfinityFoundations D

Extends `TopologicalSpace D`.

LRSOD	CIC field
D1 T0	<code>t0</code>
D2 Sobriety	<code>sober</code> (unique existence expanded: $\exists y, \dots \wedge \forall y', \dots \rightarrow y' = y$)
D3 Basis	<code>basis : Nat → (D → Prop)</code> , <code>basisOpen</code> , <code>basisCpt</code> , <code>basisGen</code> , <code>basisCap</code>
D4 Bottom	<code>bot</code> , <code>botAx</code>

Countability of the basis is built into indexing by `Nat`.

3.6 A worked witness: the Sierpiński domain

Axioms that nothing satisfies are cheap, so Layer D deserves a witness. The smallest non-degenerate one is **Sierpiński space**: carrier `Bool`, with `false` as $\perp$ and `true` as $\top$, and the opens being the up-sets $\emptyset$, $\{\top\}$, $\{\perp, \top\}$ — which is exactly the condition $U \perp \rightarrow U \top$.

A note on keywords first, since this is where the vocabulary above pays off. `instance` is reserved for `class` declarations: it registers a value for instance search to find. `DInfinityFoundations` is a `structure`, so a witness is an ordinary value, introduced with `def` to name and reuse it, or with `example` to check satisfiability without adding anything to the environment. There is no third mechanism — `instance` is not “the `structure` version of `def`”, it is the `class` version.

```
def sierpTop : TopologicalSpace Bool where
  isOpen U := U false → U true
  openEmpty := fun h => h
  openFull := fun h => h
  openUnion := by
    intro _ U hU h
    obtain ⟨i, hi⟩ := h
    exact ⟨i, hU i hi⟩
  openInter := by
    intro U W hU hW h
    exact ⟨hU h.1, hW h.2⟩
```

```
def sierp : DInfinityFoundations Bool where
  toTopologicalSpace := sierpTop
  ...
```

How each axiom is discharged:

Axiom	In this space
O1–O3	up-sets are closed under the three operations; <code>openEmpty</code> is <code>False → False</code>
$\sqsubseteq$	<code>leq false true</code> holds, <code>leq true false</code> fails — a genuine two-element order

Axiom	In this space
D1 T_0	the open $\{\top\}$ separates the points, so <code>leq</code> is antisymmetric
D2 Sobriety	the irreducible closed sets are $\{\perp\}$ and the whole space, with generic points $\perp$ and $\top$
D3 Basis	<code>basis 0 = $\{\perp, \top\}$, basis (n+1) = $\{\top\}$</code> ; both compact open, closed under intersection
D4 Bottom	<code>bot := false</code> ; every open containing $\perp$ is the whole space, since opens are up-sets

Compactness has a pleasant one-line proof here, and it is D4 doing the work: any open that covers $\perp$ is already the whole space, so a *single* member of any cover suffices, and the finite subcover is indexed by `Fin 1`.

Two things this witness settles.

D1–D4 do not pin down D_∞ . A two-point space satisfies all four. The axioms characterize a *class* of spaces — pointed ω -algebraic domains — and landing on D_∞ specifically requires the tower $D_0, D_1 = [D_0 \rightarrow D_0], \dots$ and the inverse limit as constructions *on top of* D1–D4, not as consequences of them. This is the same point Part I makes about D being a residue rather than a definition of D_∞ , now visible as a counterexample.

The classical frontier is exactly D2. Sobriety needs a case split on whether $C \top$ holds, for an arbitrary predicate C , which is `lem`. Nothing else does. That claim is machine-checked rather than asserted:

```
#print axioms sierpTop           -- does not depend on any axioms
#print axioms sierp_not_leq_top_bot -- does not depend on any axioms
#print axioms sierp              -- [lem, proptext, Quot.sound]
```

Layer O and the order facts come out **outright axiom-free**. All three axioms in the full witness enter through `sober` alone: `proptext` and `Quot.sound` via `funext` on the closure equation $C = cl(\{y\})$, and `lem` via that one case split.

3.7 Master correspondence

Layer	LRSOD count	CIC realization
L	7 schemas	kernel + <code>lem</code> for L3
R	2 rules	K4, K5
S	6	<code>SetTheory</code> (10 fields + derived <code>omega</code>)
O	3	<code>TopologicalSpace</code> (4 fields + derived topology)
D	4	<code>DInfinityFoundations</code> (9 fields)

3.8 Mathlib idiom vs deep embedding: structure or class?

Every layer above is a **structure**. Mathlib would use `structure` for one of them and `class` for the other two. The deciding question is not “is this an axiom bundle?” but **is the carrier a parameter or a field?** — in the sense fixed at the start of this part.

- Carrier as a **parameter**, at most one canonical instance per type, supplied by instance search $\rightarrow$ `class`.
- Carrier as a **field**, many instances coexisting, quantified over explicitly $\rightarrow$ `structure`.

Layer	Carrier	Mathlib form	Mathlib precedent
S — SetTheory	field V	structure	Theory.ModelType, Filter α
O — TopologicalSpace D	parameter D	class	TopologicalSpace itself
D — DInfinityFoundations D	parameter D	Prop-valued class mixins	SpectralSpace

3.8.1 O is a class in Mathlib

Not “would be” — it is, with the same three axioms and the same `IsOpen : Set X → Prop` primitive (Mathlib/Topology/Defs/Basic.lean):

```
class TopologicalSpace (X : Type u) where
  protected IsOpen : Set X → Prop
  protected isOpen_univ : IsOpen univ
  protected isOpen_inter : ∀ s t, IsOpen s → IsOpen t → IsOpen (s ∩ t)
  protected isOpen_sUnion : ∀ s, (∀ t ∈ s, IsOpen t) → IsOpen (⋃₀ s)
```

A type carries one topology in a given context, so instance resolution should find it silently. That is the whole reason `class` exists. Note the field count: Mathlib states three, deriving “ $\emptyset$ is open” by applying `isOpen_sUnion` to the empty family, where Layer O’s `openEmpty` is asserted alongside `openFull`.

3.8.2 S stays a structure

Because V is a field, not a parameter. Mathlib bundles models of a first-order theory the same way, for the same reason — one wants to compare several models at once (Mathlib/ModelTheory/Bundled.lean):

```
structure ModelType where
  Carrier : Type w
  [struc : L.Structure Carrier]
  [is_model : T.Model Carrier]
  [nonempty' : Nonempty Carrier]
```

Filter α is a structure on the same grounds: many filters per type, so nothing should be inferred.

3.8.3 D would be split into Prop mixins, not bundled

This is the widest stylistic gap. Mathlib does not write `extends TopologicalSpace D`, putting the topology in a field; it takes `[TopologicalSpace D]` as an instance parameter and asserts each condition as a separate `Prop` class. `D1` and `D2` already exist verbatim, and Mathlib’s split matches this document’s (`sober = T0 + quasi-sober`):

```
-- Mathlib/Topology/Separation/Basic.lean
class T0Space (X : Type u) [TopologicalSpace X] : Prop where
  t0 : ∀ {x y : X}, Inseparable x y → x = y

-- Mathlib/Topology/Sober.lean
class QuasiSober (α : Type*) [TopologicalSpace α] : Prop where
  sober : ∀ {S : Set α}, IsIrreducible S → IsClosed S → ∃ x, IsGenericPoint x S

-- Mathlib/Topology/Spectral/Prespectral.lean
class PrespectralSpace (X : Type*) [TopologicalSpace X] : Prop where
  isTopologicalBasis : IsTopologicalBasis { U : Set X | IsOpen U ∧ IsCompact U }
```

LRSOD	Mathlib mixin
D1 T_0	<code>T0Space</code>
D2 Sobriety	<code>QuasiSober</code> (+ <code>T0Space</code> for uniqueness of the generic point)
D3 Compact-open basis	<code>PrespectralSpace</code> + <code>SecondCountableTopology</code> for countability
D4 Bottom	no topological class; arrives from the order side (<code>OrderBot</code> / <code>OrderTop</code> on the specialization order)

The aggregate shape a Mathlib-idiomatic Layer D would take already exists as `SpectralSpace` (`Mathlib/Topology/Spectral/Basic.lean`):

```
class SpectralSpace (X : Type*) [TopologicalSpace X] : Prop extends
  T0Space X, CompactSpace X, QuasiSober X, QuasiSeparatedSpace X, PrespectralSpace X
```

So the idiomatic transcription of Layer D is:

```
class DInfinitySpace (D : Type*) [TopologicalSpace D] : Prop extends
  T0Space D, QuasiSober D, PrespectralSpace D, SecondCountableTopology D
```

with pointedness carried separately on the specialization order rather than stated about τ .

3.8.4 Why this document keeps structure anyway

Three reasons, and the second is the substantive one.

1. Nothing here should be inferred. Bundling `TopologicalSpace D` as a field of a *class* invites a diamond: D could then carry two topologies, one from `[TopologicalSpace D]` and one from inside `DInfinityFoundations`, with instance search free to pick the wrong one. Mathlib dodges this by parametrizing. As a *structure*, the hazard does not arise — the topology is passed by hand, which is what a deep embedding wants.

2. Mathlib’s mixins are Prop-valued, and this file cannot afford that. Where D3 here carries `basis : Nat → (D → Prop)` as data, Mathlib states an existential and recovers the witness with choice. `QuasiSober` pays exactly that price:

```
noncomputable def IsIrreducible.genericPoint [QuasiSober  $\alpha$ ] {S : Set  $\alpha$ } (hS : IsIrreducible S)
  :  $\alpha$  :=
  (QuasiSober.sober hS.closure isClosed_closure).choose
```

`Exists.choose` is `Classical.choice`, and `noncomputable` is the visible receipt. Layer S excludes `Choice`, so the basis must remain data. And data in a *class* is only good style when the data is *uniquely determined* by the rest of the structure — the way ωSup is pinned down by the order in `OmegaCompletePartialOrder`:

```
class OmegaCompletePartialOrder ( $\alpha$  : Type*) extends PartialOrder  $\alpha$  where
   $\omega\text{Sup}$  : Chain  $\alpha$  →  $\alpha$ 
  le_ $\omega\text{Sup}$  :  $\forall c : \text{Chain } \alpha, \forall i, c\ i \leq \omega\text{Sup } c$ 
  ...
```

A countable compact-open basis is *not* uniquely determined — a space has many — so it cannot be a canonical instance. *structure* is its honest home. The *Prop*-plus-choice versus data-plus-choice-free fork is the same trade-off in both directions: Mathlib buys canonical instances with `Choice`; this document buys choice-freedom with explicit data.

3. Orientation. Layer D’s $x \sqsubseteq y := \forall U(\tau(U) \wedge x \in U \rightarrow y \in U)$ is exactly Mathlib’s `Specializes` ($x \rightsquigarrow y$, defined as $\mathcal{N} x \leq \mathcal{N} y$). But `specializationPreorder` installs the *reverse* order, $x \leq y \leftrightarrow y \rightsquigarrow x$, oriented for algebraic geometry. Under that convention D4’s $\perp$ — the point whose only open neighbourhood is D — is the order’s **top**, so the domain-theoretic reading needs `OrderDual` (or `Specialization  $\alpha^{\text{od}}$`). A deep embedding defining `leq` directly sidesteps a silent sign error.

3.9 Constructivity note (Lean practice vs LRSOD)

Principle	LRSOD S	This embedding	Mathlib route
Choice	excluded	excluded (<code>!em</code> only, not AX2)	<code>Classical.choice</code> available
Power set	only $\mathcal{P}(\omega)$	only <code>powOmega</code>	unrestricted <code>Set</code>
Separation	Δ_0 schema	$V \rightarrow \text{Prop}$ on fixed V	<code>Set</code> comprehension
D axioms	explicit	explicit structure fields (data)	<code>Prop</code> mixin classes + <code>.choose</code>

The Sierpiński witness above measures where the line actually falls rather than predicting it: everything except sobriety is axiom-free, and `#print axioms` on the completed witness reports `[!em, propext, Quot.sound]` — no `Classical.choice` anywhere.

4 References

[Scott 1972] D. S. Scott, *Continuous lattices*, LNM 274 (1972), 97–136. Tower construction and $D_\infty \cong [D_\infty \rightarrow D_\infty]$.

[Scott 1976] D. S. Scott, Data types as lattices, *SIAM J. Computing* 5 (1976). Graph model $P\omega$ and domain reflexivity in one framework.

[Scott 1980] D. S. Scott, *Relating theories of the λ -calculus*, in *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980. Neighbourhood systems (PRG-19 lineage).

[Scott 1982] D. S. Scott, Domains for denotational semantics, *ICALP 1982*, LNCS 140. Information systems.

[Munkres 2000] J. R. Munkres, *Topology*, 2nd ed., Prentice Hall, 2000. Standard O1–O3 presentation.

[Kelley 1955] J. L. Kelley, *General Topology*, Van Nostrand, 1955. Set-theoretic topology over ZFC.

[Coquand–Huet 1988] T. Coquand, G. Huet, The calculus of constructions, *Inf. Comput.* 76 (1988), 95–120. CIC foundation.

[Carneiro 2024] L. Carneiro, *Lean 4 kernel*, Lean community documentation. Typing rules K1–K7, quotients, universes.

[de Bruijn 1980] N. G. de Bruijn, A survey of the project AUTOMATH, in *To H. B. Curry*, 1980. Deep embedding methodology.

[mathlib4] The mathlib Community, *mathlib4*. Cited files: `Topology/Defs/Basic.lean` (`TopologicalSpace`), `Topology/Defs/Filter.lean` (`Specializes`, `specializationPreorder`), `Topology/Separation/Basic.lean` (`T0Space`, `specializationOrder`), `Topology/Sober.lean` (`QuasiSober`), `Topology/Spectral/Prespectral.lean` (`PrespectralSpace`), `Topology/Spectral/Basic.lean` (`SpectralSpace`), `Order/OmegaCompletePartialOrder.lean`, `Order/Filter/Defs.lean`, `ModelTheory/Bundled.lean` (`ModelType`).

A Complete Lean source

`LRSODInCIC.lean` in full (324 lines), checked by `lake build` against Lean 4 with no Mathlib dependency. The layer structures are those of Part III; the closing sections are the Sierpiński witness for D1–D4 and its axiom audit.

```

/- =====
L, R, S, O, D expressed in Lean 4's Calculus of Inductive
Constructions.

Standalone package (no Mathlib, no scott1972/1980/1982).

Strategy: a DEEP EMBEDDING. Each layer is a `structure` whose
fields are exactly the axioms of that layer, over an abstract
carrier. Lean's ambient `Set`, `Classical.choice`, and unbounded
`Type u` hierarchy are STRONGER than S was built to be (S has no
Choice, no Replacement beyond  $\omega$ , and exactly one Power Set
application). Using them directly would silently smuggle in content
S forbids. So: no `import Mathlib`, and `Classical.choice` is never
invoked.
===== -/

/- -----
L, R - Logic and inference rules

Inherent in CIC's kernel (typing rules K1-K7, definitional
equality E1-E10) - EXCEPT L3 (classical contraposition), since bare
CIC is intuitionistic. We add the minimum needed to recover L3:
excluded middle as a single local axiom - NOT `Classical.choice`.
----- -/

axiom lem :  $\forall (p : \text{Prop}), p \vee \neg p$ 

theorem L3 (p q : Prop) : ( $\neg q \rightarrow \neg p$ )  $\rightarrow$  (p  $\rightarrow$  q) := by
  intro h hp
  cases lem q with
  | inl hq => exact hq
  | inr hnq => exact absurd hp (h hnq)

-- R1 (modus ponens) = function application; R2 (generalization) =  $\lambda$ -abstraction.

/- -----
S - Set theory (Z minus Choice minus Replacement,  $V_{\{\omega+2\}}$ )
----- -/

structure SetTheory where
  V : Type
  mem : V  $\rightarrow$  V  $\rightarrow$  Prop

  -- S1 Extensionality
  ext :  $\forall x y : V, (\forall z, \text{mem } z \ x \leftrightarrow \text{mem } z \ y) \rightarrow x = y$ 

  -- S2 Pairing
  pair : V  $\rightarrow$  V  $\rightarrow$  V
  pairAx :  $\forall x y z : V, \text{mem } z \ (\text{pair } x \ y) \leftrightarrow (z = x \vee z = y)$ 

  -- S3 Union
  union : V  $\rightarrow$  V
  unionAx :  $\forall x z : V, \text{mem } z \ (\text{union } x) \leftrightarrow \exists y, \text{mem } y \ x \wedge \text{mem } z \ y$ 

  -- S5 Separation ( $\Delta_0$ -in- $\in$  schema, rendered as `V  $\rightarrow$  Prop` on fixed V)
  sep : (V  $\rightarrow$  Prop)  $\rightarrow$  V  $\rightarrow$  V
  sepAx :  $\forall (\varphi : V \rightarrow \text{Prop}) (x z : V), \text{mem } z \ (\text{sep } \varphi \ x) \leftrightarrow (\text{mem } z \ x \wedge \varphi \ z)$ 

  succ : V  $\rightarrow$  V
  succAx :  $\forall x z : V, \text{mem } z \ (\text{succ } x) \leftrightarrow (\text{mem } z \ x \vee z = x)$ 
  empty : V
  emptyAx :  $\forall z : V, \neg \text{mem } z \ \text{empty}$ 

```

```

-- S4 Infinity
omega0 : V
infOmega : mem empty omega0 ∧ ∀ y, mem y omega0 → mem (succ y) omega0

-- S6 Power Set - only  $\mathcal{P}(\omega_0)$ 
powOmega : V
powOmegaAx : ∀ y : V, mem y powOmega ↔ (∀ z, mem z y → mem z omega0)

namespace SetTheory
variable (T : SetTheory)

def sub (x y : T.V) : Prop := ∀ z, T.mem z x → T.mem z y

def isInductive (I : T.V) : Prop :=
  T.mem T.empty I ∧ ∀ y, T.mem y I → T.mem (T.succ y) I

def omega : T.V :=
  T.sep (fun z => ∀ I, T.sub I T.omega0 → T.isInductive I → T.mem z I) T.omega0

end SetTheory

/- -----
  0 - Point-set topology, general, 3 axioms
  ----- -/

structure TopologicalSpace (D : Type) where
  isOpen : (D → Prop) → Prop

  openEmpty : isOpen (fun _ => False)
  openFull : isOpen (fun _ => True)

  openUnion : ∀ {ι : Type} (U : ι → D → Prop),
    (∀ i, isOpen (U i)) → isOpen (fun x => ∃ i, U i x)

  openInter : ∀ (U W : D → Prop), isOpen U → isOpen W →
    isOpen (fun x => U x ∧ W x)

/- -----
  D -  $D_\infty$ -specific axioms, stated purely in 0's vocabulary
  ----- -/

namespace TopologicalSpace
variable {D : Type} (T : TopologicalSpace D)

def leq (x y : D) : Prop := ∀ U, T.isOpen U → U x → U y

def isClosed (C : D → Prop) : Prop := T.isOpen (fun x => ¬ C x)

def clSingleton (y : D) : D → Prop := fun x => T.leq x y

def compact (K : D → Prop) : Prop :=
  ∀ {ι : Type} (U : ι → D → Prop),
    (∀ i, T.isOpen (U i)) →
    (∀ x, K x → ∃ i, U i x) →
    ∃ (n : Nat) (f : Fin n → ι), ∀ x, K x → ∃ j, U (f j) x

def irreducible (C : D → Prop) : Prop :=
  (∃ x, C x) ∧
  ¬ ∃ C1 C2,
    T.isClosed C1 ∧ T.isClosed C2 ∧

```



```

    (∀ x, C1 x → C x) ∧ (∀ x, C2 x → C x) ∧
    (∀ x, C x → C1 x ∨ C2 x) ∧
    (∃ x, C x ∧ ¬ C1 x) ∧ (∃ x, C x ∧ ¬ C2 x)

end TopologicalSpace

structure DInfinityFoundations (D : Type) extends TopologicalSpace D where
  t0 : ∀ x y : D, (∀ U, isOpen U → (U x ↔ U y)) → x = y

  sober : ∀ C : D → Prop,
    toTopologicalSpace.isClosed C →
    toTopologicalSpace.irreducible C →
    ∃ y, C = toTopologicalSpace.clSingleton y ∧
    ∀ y', C = toTopologicalSpace.clSingleton y' → y' = y

  basis      : Nat → (D → Prop)
  basisOpen  : ∀ n, isOpen (basis n)
  basisCpt   : ∀ n, toTopologicalSpace.compact (basis n)
  basisGen   : ∀ U, isOpen U → ∀ x, U x → ∃ n, basis n x ∧ (∀ y, basis n y → U y)
  basisCap   : ∀ m n, ∃ k, ∀ x, (basis m x ∧ basis n x) ↔ basis k x

  bot       : D
  botAx     : ∀ U, isOpen U → U bot → ∀ x, U x

/- -----
  A worked witness: the Sierpiński domain  $\perp \sqsubseteq \top$ 

  `instance` is reserved for `class` declarations. `DInfinityFoundations`
  is a `structure`, so a witness is an ordinary value: `def` to name it
  and reuse it (as here), or `example` to check satisfiability without
  adding anything to the environment.

  Carrier `Bool`, with `false` as  $\perp$  and `true` as  $\top$ . The opens are the
  up-sets -  $\emptyset$ ,  $\{\top\}$ ,  $\{\perp, \top\}$  - which is exactly the condition
   $U \perp \rightarrow U \top$ . This is Sierpiński space, the smallest non-degenerate
  Scott domain, and it satisfies D1-D4 in full.

  Two things this witness makes concrete. First, D1-D4 axiomatize a
  *class* of spaces (pointed  $\omega$ -algebraic domains) rather than pinning
  down  $D_\infty$ : reaching  $D_\infty$  specifically needs the tower and the inverse
  limit, which are constructions on top of D1-D4, not consequences of
  them. Second, the axiom audit at the end localizes the classical
  frontier - sobriety (D2) is the only field whose proof reaches for
  `lem`; 0, D1, D3 and D4 are verified constructively.
  ----- -/

def sierpTop : TopologicalSpace Bool where
  isOpen U := U false → U true
  openEmpty := fun h => h
  openFull  := fun h => h
  openUnion := by
    intro _ U hU h
    obtain ⟨i, hi⟩ := h
    exact ⟨i, hU i hi⟩
  openInter := by
    intro U W hU hW h
    exact ⟨hU h.1, hW h.2⟩

/-- Every open containing  $\perp$  is the whole space: `botAx` in usable form. -/
theorem sierp_up {U : Bool → Prop} (hU : sierpTop.isOpen U) (h : U false) :
  ∀ x, U x := by
  intro x

```

```

cases x
· exact h
· exact hU h

/--  $\{ \top \}$  is open: the separating open witnessing  $T_0$ . -/
theorem sierp_isOpen_top : sierpTop.isOpen (fun b => b = true) := fun _ => rfl

/-- Everything is below  $\top$ . -/
theorem sierp_leq_top (x : Bool) : sierpTop.leq x true := by
  intro U hU h
  cases x
  · exact hU h
  · exact h

theorem sierp_leq_bot_bot : sierpTop.leq false false := fun _ _ h => h

/--  $\top$  is not below  $\perp$  - the order is non-trivial, unlike a  $T_1$  space. -/
theorem sierp_not_leq_top_bot :  $\neg$  sierpTop.leq true false := by
  intro h
  exact absurd (h _ sierp_isOpen_top rfl) (by decide)

def sierp : DInfinityFoundations Bool where
  toTopologicalSpace := sierpTop

-- D1: the open  $\{ \top \}$  separates  $\perp$  from  $\top$ , so  $\text{leq}$  is antisymmetric.
t0 := by
  intro x y h
  cases x <;> cases y
  · rfl
  · exact absurd ((h _ sierp_isOpen_top).mpr rfl) (by decide)
  · exact absurd ((h _ sierp_isOpen_top).mp rfl) (by decide)
  · rfl

-- D2: the two irreducible closed sets are  $\{ \perp \}$  and the whole space,
-- with generic points  $\perp$  and  $\top$  respectively.
sober := by
  intro C hC hirr
  obtain ⟨w, hw⟩ := hirr.1
  cases lem (C true) with
  | inl hCt =>
    have hCf : C false := by
      cases lem (C false) with
      | inl h => exact h
      | inr h => exact absurd hCt (hC h)
    refine ⟨true, ?_, ?_⟩
    · funext x
      refine propext ⟨fun _ => sierp_leq_top x, fun _ => ?_⟩
      cases x
      · exact hCf
      · exact hCt
    · intro y' hy'
      cases y' with
      | false => exact absurd (cast (congrFun hy' true) hCt) sierp_not_leq_top_bot
      | true => rfl
  | inr hCt =>
    have hCf : C false := by
      cases w
      · exact hw
      · exact absurd hw hCt
    refine ⟨false, ?_, ?_⟩
    · funext x
      cases x
      · exact propext ⟨fun _ => sierp_leq_bot_bot, fun _ => hCf⟩

```

```

      · exact propext ⟨fun h => absurd h hCt, fun h => absurd h sierp_not_leq_top_bot⟩
    · intro y' hy'
      cases y' with
      | false => rfl
      | true  => exact absurd (cast (congrFun hy' true).symm (sierp_leq_top true)) hCt

-- D3: basis 0 = {⊥, ⊤}, basis (n+1) = {⊤}. Both compact open, and the
-- family is closed under intersection.
basis := fun n =>
  match n with
  | 0      => fun _ => True
  | _ + 1 => fun b => b = true

basisOpen := by
  intro n
  cases n
  · exact fun h => h
  · exact fun _ => rfl

basisCpt := by
  intro n / U hU hcov
  cases n with
  | zero =>
    -- one open suffices: whichever open covers ⊥ is already everything
    obtain ⟨i, hi⟩ := hcov false trivial
    exact ⟨1, fun _ => i, fun x _ => ⟨0, sierp_up (hU i) hi x⟩⟩
  | succ _ =>
    obtain ⟨i, hi⟩ := hcov true rfl
    refine ⟨1, fun _ => i, ?_⟩
    intro x hx
    cases x
    · exact Bool.noConfusion hx
    · exact ⟨0, hi⟩

basisGen := by
  intro U hU x hx
  cases x with
  | false => exact ⟨0, trivial, fun y _ => sierp_up hU hx y⟩
  | true  =>
    refine ⟨1, rfl, ?_⟩
    intro y hy
    cases y
    · exact absurd hy (by decide)
    · exact hx

basisCap := by
  intro m n
  cases m with
  | zero =>
    cases n with
    | zero  => exact ⟨0, fun _ => ⟨fun _ => trivial, fun _ => ⟨trivial, trivial⟩⟩⟩
    | succ k => exact ⟨k + 1, fun _ => ⟨fun h => h.2, fun h => ⟨trivial, h⟩⟩⟩
  | succ k =>
    cases n with
    | zero  => exact ⟨k + 1, fun _ => ⟨fun h => h.1, fun h => ⟨h, trivial⟩⟩⟩
    | succ _ => exact ⟨k + 1, fun _ => ⟨fun h => h.1, fun h => ⟨h, h⟩⟩⟩

-- D4: ⊥ is the generic point.
bot := false
botAx := fun _ hU h x => sierp_up hU h x

/-- The order is genuinely two-element: ⊥ ⊆ ⊤ but not conversely. -/
example : sierpTop.leq false true ∧ ¬ sierpTop.leq true false :=

```

```

⟨sierp_leq_top false, sierp_not_leq_top_bot⟩

-- Axiom audit. Layer 0 and the order facts are outright axiom-free; all
-- three axioms in the full witness enter through `sober` alone - `propext`
-- and `Quot.sound` via `funext`/`propext` on the closure equation, and `lem`
-- for the case split on  $C \top$ . D2 is the classical frontier here.
#print axioms sierpTop           -- does not depend on any axioms
#print axioms sierp_not_leq_top_bot -- does not depend on any axioms
#print axioms sierp             -- [lem, propext, Quot.sound]

```